Step 2 — Every Run Saved to a Database and Findable by ID — Solution

> Back to this step · Hint 1 · Hint 2 · Solution

Story — Step 2: after Step 1 a run's result lives only in the HTTP reply — once it's sent, it's gone. This step saves every run to SQLite (Doc12): a `"started"` row **before** the team runs, then `"completed"` or `"failed"` after, so even a crashed run leaves an honest record. `GET /runs/{id}` reads one back, and a request ID ties each request's log lines together. **If not:** "what happened on that run yesterday?" has no answer.

After this step the project runs: every `POST /run-task` saves a row, and `GET /runs/1` shows it. Files not shown here don't change.

The shared change

Changed from Step 1: the reply now carries the saved run's ID and status.

```
# src/contentforge/api/schemas.py
from pydantic import BaseModel, Field

class TaskRequest(BaseModel):
    # why: the same input as Project 4's "Topic:" prompt, now as JSON
    # how: "" or a 5,000-character topic gets a 422 before the route
    # even runs — a long topic would only cost money
    topic: str = Field(min_length=1, max_length=200)

class TaskResponse(BaseModel):
    # new in Step 2: the saved run's ID and status
    run_id: int
    status: str
    # why: the client must see whether the Reviewer approved the
    # article — never pass a rejected draft off as finished
    accepted: bool
    reason: str
    article: str
```

> Back to this step · Hint 1 · Hint 2 · Solution

Solution — a `db/` package, a checked run shape, and a request ID

Story — `db/database.py`: every SQL statement behind a small named function — Doc12's `runs_db.py` and Build Task, unchanged except for `DB_PATH`. Each write has its own `with conn:`, so the `"started"` row is saved on its own. `?` placeholders keep a topic like `x'); DROP TABLE runs; --` plain text. (Also create an empty `db/__init__.py`.) **If not:** SQL would be scattered through the routes, and a crashed run could vanish or stay `"started"` forever.

```
# src/contentforge/db/database.py
import sqlite3
from datetime import datetime, timezone

# how: a path from the repo root, like facts.json and the prompts
DB_PATH = "runs.db"

# why: Doc12's runs table — input, output, the agent log, a status
# that moves "started" -> "completed" / "failed", and a UTC timestamp
CREATE_RUNS_TABLE = """
    CREATE TABLE IF NOT EXISTS runs (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        input TEXT,
        output TEXT,
        log TEXT,
        status TEXT,
        created_at TEXT
    )
"""

def get_connection():
    # how: check_same_thread=False — FastAPI runs plain def routes in
    # a thread pool, so the connection is used from other threads
    conn = sqlite3.connect(DB_PATH, check_same_thread=False)
    # how: rows behave like dicts — row["input"] works
    conn.row_factory = sqlite3.Row
    conn.execute(CREATE_RUNS_TABLE)
    return conn

def insert_started_run(conn, topic):
    now = datetime.now(timezone.utc).isoformat()
    # why: its own transaction — the "started" row is saved right away,
    # so a crash later still leaves a record to look up
```

```
    with conn:
        cursor = conn.execute(
            "INSERT INTO runs (input, status, created_at) VALUES (?, ?, ?)",
            (topic, "started", now),
        )
    # how: the id SQLite just gave the new row
    return cursor.lastrowid

def finish_run(conn, run_id, output, log_text):
    with conn:
        conn.execute(
            "UPDATE runs SET output = ?, log = ?, status = ? WHERE id = ?",
            (output, log_text, "completed", run_id),
        )

def fail_run(conn, run_id):
    with conn:
        conn.execute(
            "UPDATE runs SET status = ? WHERE id = ?",
            ("failed", run_id),
        )

def fetch_run(conn, run_id):
    # why: ? placeholders — a value is never pasted into the SQL text
    sql = "SELECT * FROM runs WHERE id = ?"
    row = conn.execute(sql, (run_id,)).fetchone()
    # when: no row with this id — the route turns None into a 404
    if row is None:
        return None
    return dict(row)
```

Story — `db/models.py`: one saved run's shape, used as `GET /runs/{id}`'s `response_model` (Doc12's `RunRecord`). **If not:** a column added later — say, a cost figure — would leak to every client.

```
# src/contentforge/db/models.py
from pydantic import BaseModel

# why: one saved run's shape, checked on the way out — a column added
# to the table later never leaks to the client by accident
class Run(BaseModel):
    id: int
    input: str
    output: str | None     # None while the run is "started" or "failed"
    log: str | None
```

```
    status: str
    created_at: str
```

Changed from Step 1: `routes.py` opens the connection, saves the run around the team's work, puts the request ID on every log line, and adds `GET /runs/{run_id}` with a 404.

```
# src/contentforge/api/routes.py
from fastapi import APIRouter, HTTPException, Request

from contentforge.api.schemas import TaskRequest, TaskResponse
# new in Step 2: the database helpers and the run shape
from contentforge.db.database import get_connection, insert_started_run
from contentforge.db.database import finish_run, fail_run, fetch_run
from contentforge.db.models import Run
from contentforge.pipeline import run_pipeline
from contentforge.utils.logger import get_logger

router = APIRouter()
# why: one connection, opened once when the app starts (Doc12)
# new in Step 2
conn = get_connection()

# why: the one message a client ever sees for a server-side failure
INTERNAL_ERROR = "Internal error. Please try again later."

@router.get("/health")
def health() -> dict:
    return {"status": "ok"}

# how: a plain def route — the team's model calls block, so FastAPI
# runs this in its thread pool and other requests keep moving
@router.post("/run-task", response_model=TaskResponse)
# changed in Step 2: + http_request, to read the request ID
def run_task(request: TaskRequest, http_request: Request) -> TaskResponse:
    logger = get_logger(__name__)
    # new in Step 2: the ID made by the middleware
    request_id = http_request.state.request_id
    # why: "   " passes Pydantic (it IS 3 characters) — our own 400
    if not request.topic.strip():
        raise HTTPException(status_code=400, detail="topic cannot be empty")

    # new in Step 2: saved as "started" BEFORE the team runs
    run_id = insert_started_run(conn, request.topic)
    logger.info("[%s] run %s started", request_id, run_id)
    try:
        outcome = run_pipeline(request.topic)
```

```
    except Exception:
        # new in Step 2
        fail_run(conn, run_id)
        # how: the full traceback goes to the server log only
        # changed in Step 2: the log line now names the request and the run
        logger.exception("[%s] run %s failed", request_id, run_id)
        raise HTTPException(status_code=500, detail=INTERNAL_ERROR)

    # new in Step 2: the routing steps become the log column
    log_text = "\n".join(outcome["steps"])
    finish_run(conn, run_id, outcome["article"], log_text)
    # changed in Step 2: same log line, plus the request and run IDs
    logger.info("[%s] run %s completed: %s",
                request_id, run_id, outcome["reason"])
    return TaskResponse(
        run_id=run_id,
        status="completed",
        accepted=outcome["accepted"],
        reason=outcome["reason"],
        article=outcome["article"],
    )

# new in Step 2: look one run up by ID
@router.get("/runs/{run_id}", response_model=Run)
def get_run(run_id: int):
    run = fetch_run(conn, run_id)
    if run is None:
        raise HTTPException(status_code=404, detail="run not found")
    return run
```

Changed from Step 1: `main.py` adds Doc12's request-ID middleware.

```
# src/contentforge/api/main.py
# new in Step 2
import uuid

from fastapi import FastAPI, Request

from contentforge.api.routes import router
from contentforge.config import load_config

# why: fail at startup, not on the first real request — load_config()
# reads .env, and raises MissingConfigError if OPENAI_API_KEY is
# missing (Doc01's check, Doc12's "fail loudly" container rule)
load_config()
```

```
app = FastAPI(title="ContentForge Pro")
app.include_router(router)

# why: one short ID per request, made in one place for every route
# new in Step 2: runs around EVERY request
@app.middleware("http")
async def add_request_id(request: Request, call_next):
    request_id = str(uuid.uuid4())[:8]
    # how: request.state carries the ID into the route
    request.state.request_id = request_id
    response = await call_next(request)
    # why: the client can quote this ID when reporting a problem
    response.headers["X-Request-ID"] = request_id
    return response
```

Changed from Step 1: the same tests, with the same names, now also check the saved rows — a valid run is saved as `"completed"`, a bad body writes nothing, a crash is saved as `"failed"`, a wrong ID is a 404, and every reply has an `X-Request-ID`.

```
# tests/api/test_routes.py
# The API's own behavior — status codes and replies — with Project 4's
# team FAKED: no model calls, no cost, same every time.
# Run from the repo root: pytest tests/api -v
# The app still refuses to start without OPENAI_API_KEY — any value
# works here, because no real model is ever called.
from fastapi.testclient import TestClient

from contentforge.api import routes
from contentforge.api.main import app

client = TestClient(app)

def fake_run_pipeline(topic: str) -> dict:
    # why: a fixed answer instead of the team's real model calls
    return {
        "article": "E-Bikes Explained\n\nUp to 100 km. [battery-review]",
        "accepted": True,
        "reason": "reviewer accepted the draft",
        "steps": [
            "research_agent: no research yet",
            "END: reviewer accepted the draft",
        ],
    }

def broken_run_pipeline(topic: str) -> dict:
    # why: a crash on command, to test the 500 path
```

```
    raise RuntimeError("simulated internal failure")

def use_fake_team(monkeypatch, fake) -> None:
    # how: swaps routes.run_pipeline for the fake, for this test only
    monkeypatch.setattr(routes, "run_pipeline", fake)

# new in Step 2: count saved rows
def row_count() -> int:
    sql = "SELECT COUNT(*) FROM runs"
    return routes.conn.execute(sql).fetchone()[0]

def failed_count() -> int:
    sql = "SELECT COUNT(*) FROM runs WHERE status = ?"
    return routes.conn.execute(sql, ("failed",)).fetchone()[0]

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json() == {"status": "ok"}

def test_valid_request_is_answered(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 200
    body = response.json()
    assert body["status"] == "completed"
    assert body["accepted"] is True
    assert "[battery-review]" in body["article"]

    # new in Step 2: read the saved run back
    saved = client.get("/runs/" + str(body["run_id"])).json()
    assert saved["input"] == "electric bikes"
    assert saved["status"] == "completed"
    assert "END: reviewer accepted the draft" in saved["log"]

def test_bad_body_is_422(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    # new in Step 2
    before = row_count()
    response = client.post("/run-task", json={})
    assert response.status_code == 422
    assert row_count() == before

def test_blank_topic_is_400(monkeypatch):
    use_fake_team(monkeypatch, fake_run_pipeline)
    response = client.post("/run-task", json={"topic": "   "})
    assert response.status_code == 400
```

```
    assert response.json() == {"detail": "topic cannot be empty"}

def test_internal_error_is_a_plain_500(monkeypatch):
    use_fake_team(monkeypatch, broken_run_pipeline)
    # new in Step 2
    before = failed_count()
    response = client.post("/run-task", json={"topic": "electric bikes"})
    assert response.status_code == 500
    # why: the client never sees "RuntimeError" or a traceback
    expected = {"detail": "Internal error. Please try again later."}
    assert response.json() == expected
    # how: the crashed run is still saved, honestly, as "failed"
    assert failed_count() == before + 1

# new in Step 2
def test_unknown_run_is_404():
    response = client.get("/runs/999999")
    assert response.status_code == 404

# new in Step 2
def test_every_reply_has_a_request_id():
    response = client.get("/health")
    assert len(response.headers["X-Request-ID"]) == 8
```

Run it:

```
pytest tests/api -v
uvicorn contentforge.api.main:app --reload
```

```
tests/api/test_routes.py::test_health PASSED
tests/api/test_routes.py::test_valid_request_is_answered PASSED
tests/api/test_routes.py::test_bad_body_is_422 PASSED
tests/api/test_routes.py::test_blank_topic_is_400 PASSED
tests/api/test_routes.py::test_internal_error_is_a_plain_500 PASSED
tests/api/test_routes.py::test_unknown_run_is_404 PASSED
tests/api/test_routes.py::test_every_reply_has_a_request_id PASSED
```

Then `curl localhost:8000/runs/1` (shown wrapped to fit the page — really one line):

```
{"id":1,"input":"electric bikes","output":"E-Bikes: How Far ...",
 "log":"research_agent: no research yet\nanalysis_agent: research done,
 no findings yet\n...\nEND: reviewer accepted the draft",
 "status":"completed","created_at":"2026-01-01T00:00:00.000000+00:00"}
```

Server log for one request (every line has the same ID):

```
2026-01-01 00:00:00,000 contentforge.api.routes INFO [9ee100c1] run 1 started
2026-01-01 00:00:09,000 contentforge.api.routes INFO [9ee100c1] run 1 compl
eted: reviewer accepted the draft
```

(The second line is really one line — shown wrapped.) The tests write to the same `runs.db` as the app; it's in `.gitignore`, and you can delete it any time.